

Задания на автомат по курсу «Одноплатные компьютеры и embedded-разработка»

Каждое задание охватывает несколько лабораторных работ курса и включает разработку приложения на языке Python для ALT Linux (GUI — на выбор студента: Tkinter, PyQt, Gtk и др.; консольное — для задания №7). Транспорт между компонентами — протокол MQTT, брокер Mosquitto на Lichee RV Dock.

Задание №5 (Светодиодный проигрыватель мелодий) вынесено в отдельный файл `auto_melody.md`.

Задание 1. Метеостанция — сбор и визуализация данных с датчика BME280

Охват лабораторных: №7 (SPI), №9 (I2C OLED), №12 (Arduino + SPI + BME280), №13 (MQTT).

Описание

Собрать распределённую систему, в которой данные с датчика BME280 (температура, влажность, давление) передаются от Arduino к одноплатнику Lichee, далее через MQTT на ПК и отображаются в реальном времени в графическом интерфейсе.

Цепочка

```
BME280 —I2C—► Arduino Uno —SPI(slave)—► Lichee RV Dock —MQTT—► ПК (GUI)
```

Требования

- 1. Arduino + датчик (лаб. №12).** Прошивка Arduino читает BME280 по I2C (библиотека GyverBME280), передаёт три значения float (температура °C, влажность %, давление гПа) по SPI в режиме slave на Lichee. Протокол — серия одnobайтовых транзакций.
- 2. Lichee — SPI-приём + MQTT-публикация (лаб. №7, №13).** Python-скрипт на Lichee принимает данные от Arduino через `/dev/spidev1.0`, распаковывает `struct.unpack("<fff", ...)` и публикует их в MQTT-топик `sensors/bme280` в формате JSON:

```
{"temperature": 25.1, "humidity": 48.3, "pressure": 1004.5, "timestamp": "2026-05-04T12:34:56"}
```

- 3. I2C OLED (лаб. №9).** Дополнительно — дублировать данные на OLED-экран SSD-1306 через I2C на Lichee (три строки: температура, влажность, давление).

4. **ПК — GUI-приложение.** Python-приложение, подключающееся к MQTT-брокеру на Lichee и отображающее:

- **Текущие значения** трёх параметров — крупным шрифтом в отдельных блоках (например, «температура» подписана, значение выделено цветом)
- **Три графика** (`matplotlib`, встроенный в GUI) с историей за последние 60 секунд (тип графика `line` или `plot`, ось X — время, ось Y — значение)
- **Статус подключения:** индикатор (зелёный/красный кружок) — подключён ли датчик, получает ли система данные в данный момент. Статус менять по тайм-ауту: если данные не поступали более 5 секунд — «нет данных»

Ключевые точки проверки

- Физически подключённый BME280 выдаёт реалистичные значения
- При отключении датчика от Arduino GUI показывает «нет данных» (красный индикатор)
- При отключении Arduino от Lichee — аналогично
- Графики обновляются плавно, история не теряется при свёртывании окна

Задание 2. Пульт управления светодиодами — remote control через MQTT и SPI

Охват лабораторных: №11 (Arduino GPIO, прерывания), №12 (SPI-обмен), №13 (MQTT).

Описание

Через графический интерфейс на ПК управлять светодиодами на мезонинной плате Arduino, передавая команды по цепочке MQTT → Lichee → SPI → Arduino.

Цепочка

ПК (GUI) —MQTT—► Lichee RV Dock —SPI(master)—► Arduino Uno —GPIO—► LED-мезонин

Требования

1. **Arduino — приём команд по SPI (лаб. №11, №12).** Прошивка работает в режиме SPI-slave и управляет светодиодами на пинах 3, 5, 6, 9, 10 (мезонинная плата). Команды приходят от Lichee-мастера. Протокол команд студент проектирует самостоятельно (например: байт команды + байт данных; или 5 бит, соответствующих пяти светодиодам).

Поддерживаемые операции для каждого светодиода:

- включить / выключить (индивидуально)
- «бегущий огонёк» вперёд (пины 3→10→3)
- «бегущий огонёк» назад (пины 10→3→10)
- остановить анимацию

Скорость бегущего огонька задаётся параметром (задержка в миллисекундах).

2. **Lichee — MQTT-подписчик + SPI-мастер (лаб. №13).** Python-скрипт подписывается на MQTT-топик `led/command`, получает JSON-команды и транслирует их в SPI-транзакции к Arduino.

Формат JSON-команды (примеры):

```
{"pin": 3, "action": "on"}
{"pin": 5, "action": "off"}
{"action": "animation", "direction": "forward", "delay_ms": 150}
{"action": "stop"}
```

3. **ПК — GUI-приложение.** Окно с элементами управления:

- 5 тумблеров (checkboxbutton/toggle) для индивидуальных светодиодов с подписями «LED 3», «LED 5», ...
- Кнопка «Бегущий вперёд» и кнопка «Бегущий назад»
- Слайдер задержки анимации (50–500 мс)
- Кнопка «Стоп»
- При изменении любого элемента GUI публикуется соответствующая MQTT-команда

Ключевые точки проверки

- Каждый тумблер индивидуально включает/выключает свой светодиод
- Бегущий огонёк работает в обоих направлениях
- Смена задержки на слайдере меняет скорость анимации
- «Стоп» останавливает анимацию и оставляет текущее состояние светодиодов

Задание 3. Системный монитор одноплатника — dashboard

Охват лабораторных: №3 (кросс-компиляция), №8 (hasher, удалённая разработка), №13 (MQTT).

Описание

Создать дашборд (панель мониторинга), отображающий в реальном времени системные метрики одноплатника Lichee: загрузку CPU, использование RAM, занятость корневого раздела, сетевой трафик. Метрики собираются на Lichee и публикуются по MQTT. GUI на ПК отображает их в виде приборных панелей и графиков.

Цепочка

```
Lichee RV Dock (/proc, /sys) —MQTT—► ПК (GUI)
```

Требования

1. **Lichee — сборщик метрик (лаб. №13).** Python-скрипт-издатель, публикующий в MQTT-топик `lichee/stats` JSON-пакет раз в 2 секунды:

```
{
  "cpu_percent": 12.3,
  "ram_percent": 45.7,
  "disk_percent": 32.1,
  "rx_bytes_per_sec": 1024,
  "tx_bytes_per_sec": 512,
  "uptime_seconds": 12345,
  "hostname": "lichee-rv",
  "kernel": "6.16.0"
}
```

Источники данных:

- CPU: `/proc/stat`
- RAM: `/proc/meminfo`
- Диск: `os.statvfs('/')`
- Сеть: `/sys/class/net/<интерфейс>/statistics/rx_bytes, tx_bytes` (дельта между опросами)
- Uptime: `/proc/uptime`
- Hostname: `socket.gethostname()`
- Версия ядра: `os.uname().release`

2. **Дополнительно (лаб. №3, №8).** Написать сборщик метрик на языке C с использованием только системных вызовов и файлов `/proc`, `/sys`. Скомпилировать под RISC-V (кросс-компиляция с x86_64). По быстрдействию сравнить с Python-версией.

3. **ПК — GUI-приложение.** Интерфейс дашборда содержит:

- **4 «приборных панели»** (gauge / стрелочный индикатор) для CPU, RAM, диска, сети. Каждая панель показывает текущее значение в процентах и имеет цветовое кодирование (зелёный < 50%, жёлтый 50–80%, красный > 80%). Допускается использовать `matplotlib` с кастомными шкалами или рисовать на `Canvas`.
- **Строка статуса** в нижней части окна: uptime, hostname, версия ядра.
- **График истории** (опционально, `matplotlib`): загрузка CPU и RAM за последние 60 секунд на одном графике с двумя осями Y.
- **Управление:** кнопка «Обновить сейчас» — публикует команду в MQTT-топик `lichee/cmd` (`{"action": "refresh"}`), по которой Lichee немедленно публикует свежий пакет метрик (вне очереди по таймеру). На Lichee нужно реализовать подписку на этот топик и реакцию на команду.

Ключевые точки проверки

- Все четыре панели обновляются и показывают адекватные значения

- При нагрузке на Lichee (запуск **stress** или **dd**) значения меняются
- Кнопка «Обновить сейчас» вызывает немедленную публикацию
- Строка статуса показывает реальные uptime и hostname

Задание 4. Генератор сигналов с управлением по MQTT — практическая верификация анализатором

Охват лабораторных: №10 (libgpiod2, логический анализатор, PulseView), №13 (MQTT).

Описание

С графического интерфейса на ПК задать параметры цифрового сигнала (меандра). Lichee генерирует сигнал на GPIO-пине. Студент подключает логический анализатор к этому пину, захватывает сигнал в PulseView и подтверждает соответствие заданным параметрам.

Цепочка

```
ПК (GUI) —MQTT—► Lichee (libgpiod2) —GPIO—► Логический анализатор  
(fx2lafw) —USB—► ПК (PulseView)
```

Требования

1. **Lichee — генератор сигнала (лаб. №10).** Python-скрипт подписывается на MQTT-топик **gpio/control**, принимает JSON-команды и управляет GPIO-пином (например, PE16, линия 144) через **libgpiod2**.

Формат JSON-команды:

```
{"action": "start", "gpio_line": 144, "freq_hz": 1000, "duty_percent": 50}  
{"action": "stop"}
```

После получения **"start"** скрипт в отдельном потоке генерирует меандр на указанном пине с заданной частотой и скважностью. При **"stop"** поток останавливается, пин переводится в низкий уровень. При повторном **"start"** без **"stop"** параметры обновляются без разрыва сигнала.

2. **ПК — GUI-приложение.** Окно с элементами управления:

- **Выпадающий список** (combobox) с доступными GPIO-линиями Lichee (минимум 3: 144, 145, 146 — пины PE16, PE17, PE18; номера взять из распиновки платы). Или текстовое поле для ввода номера линии.
- **Поле ввода частоты** (Гц) — от 1 до 100 000 Гц, с проверкой корректности ввода. Значение по умолчанию: 1000 Гц.
- **Слайдер или поле ввода скважности** (%) — от 1 до 99%. По умолчанию: 50%.

- **Кнопка «Старт»** — отправка MQTT-команды `start` с текущими параметрами из GUI.
- **Кнопка «Стоп»** — отправка `stop`.
- **Предпросмотр сигнала** — на `Canvas` отрисовывается визуализация меандра: два периода прямоугольного сигнала с учётом заданной скважности. Подписи: длительность периода в мкс, длительность высокого и низкого уровня. Предпросмотр обновляется при каждом изменении параметров без отправки команды на Lichee (чисто клиентская графика).

3. **Верификация (лаб. №10).** Подключить логический анализатор (fx2lafw) к заданному GPIO-пину Lichee. Запустить захват в PulseView (частота дискретизации $\geq 4\times$ от частоты сигнала, но не менее 12 МГц). Измерить период и скважность сигнала и сравнить с заданными в GUI значениями. Продемонстрировать изменение сигнала при смене параметров через GUI.

Ключевые точки проверки

- Сигнал на GPIO появляется при нажатии «Старт» и пропадает при «Стоп»
- Частота и скважность, измеренные в PulseView, соответствуют заданным (погрешность в пределах 5% для частот до 10 кГц; на более высоких частотах неизбежны задержки из-за userspace-управления GPIO)
- Изменение параметров через GUI во время работы генератора обновляет сигнал без остановки
- Предпросмотр на Canvas корректно отображает два периода меандра с верными метками времени
- При вводе некорректных значений (частота 0, скважность 100%) GUI выдаёт ошибку валидации, не отправляя команду

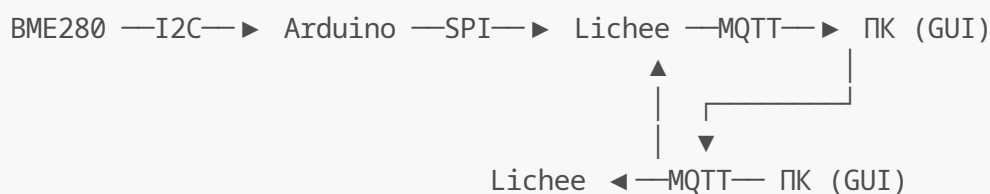
Задание 6. Система мониторинга с оповещением на I2C OLED

Охват лабораторных: №7 (SPI), №9 (I2C OLED), №12 (Arduino + BME280 + SPI), №13 (MQTT).

Описание

Расширение задания №1 (Метеостанция). Данные с датчика BME280 идут по цепочке Arduino → SPI → Lichee → MQTT → GUI на ПК. Дополнительно оператор задаёт в GUI пороговые значения для каждого параметра. При превышении порога GUI сигнализирует тревогу. Оператор жмёт кнопку «Оповестить на OLED» — по обратному каналу MQTT → Lichee на I2C-экран SSD-1306 выводится тревожное сообщение с текущим значением.

Цепочка





Требования

1. **Arduino + датчик (лаб. №12).** Прошивка идентична заданию №1: читает BME280 по I2C (GyverBME280), передаёт три float по SPI-slave на Lichee (12 однобайтовых транзакций).
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №13).** Python-скрипт идентичен заданию №1: принимает данные по SPI, публикует JSON в топик `sensors/bme280`.
3. **Lichee — подписчик на тревоги (лаб. №9, №13).** Отдельный Python-скрипт, подписывающийся на топик `alert/oled`. При получении сообщения выводит текст на SSD-1306 через I2C (luma-oled): крупным шрифтом на весь экран. Через 5 секунд автоматически возвращает на экран обычные показания (либо очищает).

Формат сообщения в `alert/oled`:

```
{"message": "ALARM!\nT=35.2 C\nLimit=30.0 C"}
```

4. **ПК — GUI-приложение.** Все элементы задания №1 плюс:
 - **Поля ввода порогов:** `T_max`, `H_max`, `P_min`, `P_max`. При старте — значения по умолчанию (например: 30°C, 70%, 950 гПа, 1050 гПа)
 - **Индикация тревоги:** при превышении порога строка соответствующего параметра подсвечивается красным и мигает. Если превышено несколько параметров — подсвечены все нарушенные
 - **Кнопка «Оповестить на OLED»:** активна только когда есть активная тревога. При нажатии публикует сообщение в `alert/oled`
 - **Сброс тревоги:** тревога сбрасывается, когда значение возвращается в пределы порога + гистерезис (например, 0.5°C для температуры — чтобы не дёргалась при значении ровно на границе)

Ключевые точки проверки

- При превышении порога GUI подсвечивает соответствующую строку красным
- Кнопка «Оповестить на OLED» активна только при тревоге; при нажатии сообщение появляется на физическом экране SSD-1306
- Через 5 секунд экран возвращается к обычному отображению
- При возврате значения в норму тревога сбрасывается с гистерезисом
- При превышении двух параметров одновременно сообщение содержит оба значения

Задание 7. Генератор загрузочных образов — консольный сборщик SD-карты

Охват лабораторных: №4 (ядро Linux), №5 (Device Tree, U-Boot, extlinux.conf), №6 (запись образа на SD-карту), №7 (SPI-ядро), №9 (I2C-ядро).

Описание

Создать консольное приложение на Python, которое из готовых компонентов автоматически собирает загрузочную SD-карту для Lichee RV Dock с заданной конфигурацией. Готовые ядра, Device Tree Blob, rootfs и загрузчик предоставляются преподавателем.

Цепочка



Готовые компоненты (предоставляются преподавателем)

Файл	Назначение
rootfs.tar.gz	Корневая файловая система ALT Linux (RISC-V)
Image	Ядро с поддержкой WiFi (базовое)
Image-spi	Ядро с WiFi + SPI (лаб. №7)
Image-i2c	Ядро с WiFi + I2C (лаб. №9)
sun20i-d1-lichee-rv-dock.dtb	Device Tree (базовый)
sun20i-d1-lichee-rv-dock-spi.dtb	Device Tree с активированным SPI
sun20i-d1-lichee-rv-dock-i2c.dtb	Device Tree с активированным I2C
u-boot-sunxi-with-spl.bin	Загрузчик U-Boot (SPL + Proper)
extlinux.conf	Шаблон конфигурации загрузчика

Скрипт `build_image.py`

Интерфейс командной строки:

```
$ python3 build_image.py --device /dev/sdX --config spi [--dry-run]
```

Параметр	Описание
--device	Путь к устройству SD-карты (обязательный)
--config	Конфигурация: <code>basic</code> (только WiFi), <code>spi</code> , <code>i2c</code> , <code>full</code> (всё)
--dry-run	Показать план действий без реальной записи

Конфигурации и соответствующие им файлы:

Конфигурация	Ядро	Device Tree	Поддерживаемые интерфейсы
basic	Image	sun20i-d1-lichee-rv-dock.dtb	WiFi
spi	Image-spi	sun20i-d1-lichee-rv-dock-spi.dtb	WiFi + SPI
i2c	Image-i2c	sun20i-d1-lichee-rv-dock-i2c.dtb	WiFi + I2C
full	Image-spi	sun20i-d1-lichee-rv-dock-spi.dtb + i2c в dtb	WiFi + SPI + I2C

Действия программы (по порядку):

- 1. Проверка.** Убедиться, что `--device` существует, является блочным устройством и не примонтирован. Если примонтирован — выдать ошибку и завершиться. В режиме `--dry-run` — только вывести план.
- 2. Разбивка.** `fdisk` создаёт таблицу разделов DOS:
 - Первый раздел: 100 МБ, тип FAT32 (LBA)
 - Второй раздел: оставшееся место, тип Linux (ext4)
- 3. Запись загрузчика.** `dd if=u-boot-sunxi-with-spl.bin of=<device> bs=1k seek=8`
- 4. Форматирование.** `mkfs.vfat -F 32 <device>1` и `mkfs.ext4 <device>2`
- 5. Копирование файлов на BOOT.** Монтирование раздела 1, копирование ядра (как `Image`), Device Tree (как `.dtb`), шаблона `extlinux.conf` (при необходимости — подстановка правильного имени ядра и dtb).
- 6. Распаковка rootfs.** Монтирование раздела 2, `tar xf rootfs.tar.gz -C <mountpoint>`.
- 7. sync** и уведомление о завершении.

Вывод программы — человекочитаемый лог каждого шага:

```
[INFO] Checking device /dev/sdb ... OK (not mounted)
[INFO] Selected config: spi
[INFO] Step 1/6: Partitioning ...
[INFO] Step 2/6: Writing U-Boot (seek=8) ...
[INFO] Step 3/6: Formatting BOOT (vfat) ...
[INFO] Step 4/6: Formatting ROOTFS (ext4) ...
[INFO] Step 5/6: Copying kernel + dtb + extlinux.conf ...
[INFO] Step 6/6: Extracting rootfs (this may take a while) ...
[INFO] Syncing ...
[INFO] Done. SD card is ready at /dev/sdb
```

Дополнительные возможности (опционально)

- **GUI-обёртка** (Tkinter) с выбором устройства из выпадающего списка, выбором конфигурации радиокнопками, прогресс-баром и встроенным логом
- **Поддержка образа в файл:** `--output image.img` вместо физического устройства (создаёт файл образа `.img`, который можно записать на карту через `dd`)
- **Валидация файлов:** перед записью проверить, что все необходимые файлы присутствуют в рабочей директории

Ключевые точки проверки

- Карта, собранная с конфигурацией `basic`, загружается, работает WiFi
- С конфигурацией `spi` — загружается, WiFi и SPI работают (проверить `ls /dev/spidev*`)
- С конфигурацией `i2c` — загружается, WiFi и I2C работают (проверить `ls /dev/i2c-*`)
- `--dry-run` выводит план, но не трогает SD-карту
- При попытке записать на примонтированное устройство — ошибка, а не порча данных

Задание 8. Система мониторинга со звуковой сигнализацией

Охват лабораторных: №7 (SPI), №12 (Arduino + BME280 + SPI), №13 (MQTT).

Описание

Расширение задания №1 (Метеостанция). Как в задании №6, но вместо I2C OLED — звуковое оповещение на ПК. При превышении порогового значения GUI автоматически воспроизводит звуковую сирену. Характер звука (длительность, прерывистость) настраивается для каждого измеряемого параметра индивидуально. Оператор может квитировать тревогу кнопкой.

Цепочка

BME280 —I2C—► Arduino —SPI—► Lichee —MQTT—► ПК (GUI + динамик)

Требования

1. **Arduino + датчик (лаб. №12).** Прошивка идентична заданию №1.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №13).** Python-скрипт идентичен заданию №1.
3. **ПК — GUI-приложение.** Все элементы задания №1 (значения + графики) плюс:
 - **Поля ввода порогов** (`T_max`, `H_max`, `P_min`, `P_max`) как в задании №6
 - **Настройки сирены для каждого параметра:**
 - Выпадающий список типа звука:
 - «прерывистый быстрый» (200 мс тон / 200 мс пауза)
 - «прерывистый медленный» (500 мс / 500 мс)
 - «непрерывный»
 - «пользовательский WAV-файл» (выбор через диалог)

- Поле выбора частоты тона (Гц): 440 (A4), 880 (A5), 1760 (A6) и т.д.
- **Автоматическое воспроизведение сирены** при превышении порога:
 - Сирена воспроизводится циклически, пока оператор не нажмёт «Отключить сирену»
 - Если превышено несколько параметров одновременно — звуки чередуются (каждый цикл — свой тип звука для своего параметра) или используется наиболее критичный (наибольшее отклонение от порога в процентах)
- **Кнопка «Отключить сирену» (квитирование):**
 - Прекращает воспроизведение до следующего превышения
 - Тревога остаётся визуальной (красная подсветка), пока значение не вернётся в норму
- **Лог тревог:** таблица в отдельной вкладке или в нижней части окна:
 - Колонки: Время, Параметр, Значение, Порог
 - Автоматическое добавление строки при каждом новом превышении
 - Кнопка «Экспорт в CSV»

Генерация звука — через стандартную библиотеку Python `wave` (без дополнительных зависимостей):

```
import wave
import struct
import subprocess

def generate_tone(freq, duration_ms, volume=0.5):
    """Генерирует WAV-файл в памяти и воспроизводит через aplay."""
    sample_rate = 44100
    n_samples = int(sample_rate * duration_ms / 1000)
    buf = b""
    for i in range(n_samples):
        t = i / sample_rate
        value = int(volume * 32767 * (1 if int(t * freq * 2) % 2 == 0 else -1))
        buf += struct.pack("<h", value)

    wav = wave.open("/tmp/alert_tone.wav", "w")
    wav.setnchannels(1)
    wav.setsampwidth(2)
    wav.setframerate(sample_rate)
    wav.writeframes(buf)
    wav.close()

    subprocess.run(["aplay", "/tmp/alert_tone.wav"], capture_output=True)
```

При выборе пользовательского WAV-файла проигрывается он через `aplay`.

Ключевые точки проверки

- При превышении порога GUI подсвечивает параметр красным и автоматически издаёт звуковую сирену
 - Характер звука разный для разных параметров (в соответствии с настройками)
 - Кнопка «Отключить сирену» прекращает звук; при повторном превышении (возврат в норму и снова превышение) — сирена включается заново
 - Лог тревог пополняется при каждом превышении; экспорт в CSV — корректный
 - При превышении двух параметров одновременно GUI корректно чередует или приоритизирует звуки
-

Порядок сдачи

1. Студент выбирает **одно** из восьми заданий (№1–№4 — в этом файле, №5 — в `auto_melody.md`)
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino `.ino`, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для С-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений